AI in Fluids - Week 1.1

AI-assisted scientific software

Lecture notes by Ehsan Roohi | FlowMLLab

1. Purpose and connection to Week 1

Week 1 introduced velocity fields, conservation of mass, finite differences and the lid-driven cavity. This supplementary lecture asks how we can establish that the software used to manipulate those fields is trustworthy. The question applies whether the implementation is written by a student, a collaborator or an AI coding assistant. A program may execute successfully and produce an attractive contour while calculating a different quantity from the one its author intended.

We study a small but scientifically meaningful component: a post-processor that computes divergence and out-of-plane vorticity from two velocity arrays. Its size allows us to examine every mathematical assumption and connect it to executable evidence. We first construct an analytic reference field, then measure finite-difference error across four grids, and finally audit the retained Week-1 cavity at Reynolds number 100. The accompanying notebook includes an intentionally incorrect proposal that must be rejected.

After working through the lecture, you should be able to specify array and derivative conventions, distinguish mathematical verification from physical validation, interpret a convergence study, and explain exactly what a passed acceptance test establishes. You should also be able to use an AI assistant within a reproducible development process while retaining responsibility for the scientific interpretation. Use any accessible coding assistant. The assessed implementation requires a real recorded interaction; ask the instructor for an alternative if none is accessible.

Student roadmap

Part A is the completed derivative example: run it, explain the grid-error trend and inspect the deliberately rejected axis bug. Part B is your assessed new function net_volume_flux. You are not writing another cavity solver. SCIENTIFIC_SPEC.md describes Part A; SPEC.md is the template you complete for Part B.

Begin with the four boundary edges: outward flow is right minus left for u and top minus bottom for v. Constant velocity has zero total flux. On the unit square, u=x and v=y give flux 2. Predict these values by hand before asking an assistant. Zero total flux does not guarantee zero local divergence.

Follow six steps: run the example; derive expected fluxes; complete and commit your specification; save the first agent patch; test and review your changes; submit code and evidence in your fork. See ASSIGNMENT.md for the exact folder map.

2. What does it mean for scientific code to be correct?

Four questions should be kept separate. First, does the program execute? An execution test can detect an unavailable package, a shape mismatch or a non-finite output. These checks are essential, but successful execution tells us little about the physical meaning of the returned numbers. Differentiation along the wrong axis of a square array, for example, can preserve the output shape.

Second, does the implementation correctly approximate its mathematical definition? This is the question of verification. A derivative operator can be checked against a function whose derivative is known analytically. Repeating the calculation on finer grids tests whether the error decreases at the rate predicted by the discretization. A single small error is less informative than a consistent error trend because cancellation can

make an incorrect method appear accurate at one resolution.

Third, does the mathematical model adequately represent the physical system for the intended use? This is the question of validation. It requires appropriate independent physical evidence and an assessment of uncertainty. Comparing a derivative with a stored cavity field is a consistency audit of the post-processor, not a new physical validation of the cavity solver. A reference produced by related numerical operations may share some of the same errors.

Fourth, can someone reconstruct the result? Reproducibility requires identified inputs, the implementation revision, the software environment, the commands and the decision criteria. A result may be reproducible yet wrong; an accurate-looking result may be impossible to reconstruct. Verification and reproducibility complement one another rather than substitute for one another [1,2].

3. Write the scientific contract before the implementation

The contract describes what the function should calculate and the evidence required to accept it. Our input consists of one-dimensional coordinate arrays x and y and two velocity arrays u and v. The storage convention is u[j,i] = u(y[j],x[i]). Thus the first array axis follows y and the second follows x. Both velocity arrays must have shape (len(y), len(x)); the coordinates must be finite and strictly increasing.

This is a scientific requirement, not merely a programming preference. The horizontal velocity component u is differentiated with respect to x in one part of the divergence, but with respect to y in one part of the vorticity. The component name and coordinate direction serve different roles. Writing out these roles before coding is often the most effective way to prevent an axis error.

The contract must state the units. Dimensional velocity derivatives have units of inverse time. If velocity and position are nondimensionalized by U and L, the corresponding derivative represents the dimensional derivative multiplied by L/U. Tolerances such as 1e-12 cannot be transferred automatically between dimensional and nondimensional calculations or between floating-point precisions. Here the cavity uses the Week-1 normalization, with unit side length and unit lid velocity, and the diagnostic uses float64 arithmetic.

Specify the region over which each metric is evaluated. Interior comparisons omit two grid layers on every side. Wall velocities are checked separately, excluding the corners because the moving lid and stationary sidewalls impose incompatible limiting velocities there. These exclusions must accompany the metric. A claim about the interior is not a claim about every boundary node.

4. Divergence, vorticity and the discrete derivative

For a two-dimensional velocity field, divergence measures local volumetric expansion and vorticity describes the rotational part of the velocity gradient. Using the same sign convention as Week 1, we define

$$\nabla \cdot \mathbf{u} = \partial_x u + \partial_y v, \qquad \omega_z = \partial_x v - \partial_y u \tag{1}$$

Positive z-vorticity corresponds to counterclockwise rotation in the x-y plane. To check the sign independently, consider rigid rotation with u = -a y and v = a x. Its divergence is zero and its vorticity is 2a. This example is useful because the expected result follows directly from differentiation and does not require a CFD solver.

For a smooth scalar field on a uniform grid, Taylor expansions about the point (j,i) give

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(x) + \mathcal{O}(h^4) \tag{2}$$

Subtracting the expansions eliminates the even derivative terms. Dividing by twice the grid spacing gives the centered difference and its leading truncation error:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + \mathcal{O}(h^4) \tag{3}$$

The leading error is proportional to the square of the spacing, provided the field is sufficiently smooth and higher-order terms are small. Halving the spacing should then reduce the error by approximately a factor of four. This expectation motivates our convergence study. It is not a guarantee for discontinuities or singular corners, which do not satisfy the smoothness assumption.

In NumPy, the required operations are conceptually du_dx = gradient(u, x, axis=1) and du_dy = gradient(u, y, axis=0), with corresponding operations for v. The implementation sets edge_order=2. Passing coordinate vectors avoids silently assuming unit spacing. Although the implementation accepts increasing nonuniform coordinates, the retained convergence evidence concerns uniform Cartesian grids; a stretched-grid claim requires another study.

5. Constructing an analytic verification field

An analytic field supplies a reference independent of the numerical derivative under test. We use the streamfunction

$$\psi(x, y) = \sin^2(\pi x)\sin^2(\pi y), \qquad (x, y) \in [0, 1]^2 \tag{4}$$

As in Week 1, define u as the y-derivative of the streamfunction and v as minus its x-derivative. Differentiating analytically gives

$$u = \psi_y = \pi \sin^2(\pi x)\sin(2\pi y)$$

$$v = -\psi_x = -\pi \sin(2\pi x)\sin^2(\pi y) \tag{5}$$

The mixed second derivatives of the smooth streamfunction cancel in the divergence, so exact divergence is zero. Every wall velocity also vanishes because the relevant sine factor is zero. This field avoids the lid-corner discontinuity of the cavity. It is a constructed reference for derivative verification, not a claim that these velocities solve the unforced steady cavity equations.

With our sign convention the exact vorticity equals minus the Laplacian of the streamfunction:

$$\omega_z = -\nabla^2\psi$$

$$= -2\pi^2[\cos(2\pi x)\sin^2(\pi y) + \sin^2(\pi x)\cos(2\pi y)] \tag{6}$$

The notebook evaluates these analytic expressions directly on each grid. Reference vorticity must not be generated by applying the same finite-difference operator to sampled velocity: doing so would test a routine against itself. Independent reference construction is particularly important when an AI assistant proposes both an implementation and its tests. Two generated routines can reproduce the same mistaken convention.

6. Quantifying error and observed convergence

Let I_h denote the retained interior points on a grid of spacing h. We compare computed vorticity with its analytic value using the discrete relative L2 error

$$E_h = \frac{\left[\sum_{(j,i)\in I_h} (\omega_{j,i}^{h} - \omega_{j,i}^{\mathrm{exact}})^2\right]^{1/2}}{\left[\sum_{(j,i)\in I_h} (\omega_{j,i}^{\mathrm{exact}})^2\right]^{1/2}} \tag{7}$$

Exact divergence is zero, so a relative error with exact divergence in the denominator would be undefined. We instead report the root-mean-square absolute divergence over the same interior. Choosing an appropriate error measure is part of the scientific method; it should not be left to an arbitrary library default.

The four grids have 17, 33, 65 and 129 points per direction, with spacings 1/16, 1/32, 1/64 and 1/128. The interior omits two layers on each grid; its physical boundary therefore moves slightly as h changes. These values describe convergence under that support convention, rather than a norm over an identical fixed interior region at every resolution.

Table 1. Analytic verification on four uniform grids; two layers are excluded from each edge.

Points per direction	Grid spacing	Relative L2 error	Divergence RMS
17	0.062500	2.550464e-02	3.059e-15
33	0.031250	6.413149e-03	6.954e-15
65	0.015625	1.605607e-03	1.481e-14
129	0.007812	4.015469e-04	2.748e-14

If the leading error behaves as C times h to the power p, taking logarithms produces a straight-line relationship. The implementation estimates p by fitting log(E_h) against log(h) using all four grids. It obtains approximately 1.9965. The familiar ratio log(E_2h/E_h)/log(2) estimates a local two-grid order, but is not the estimator used for this reported result.

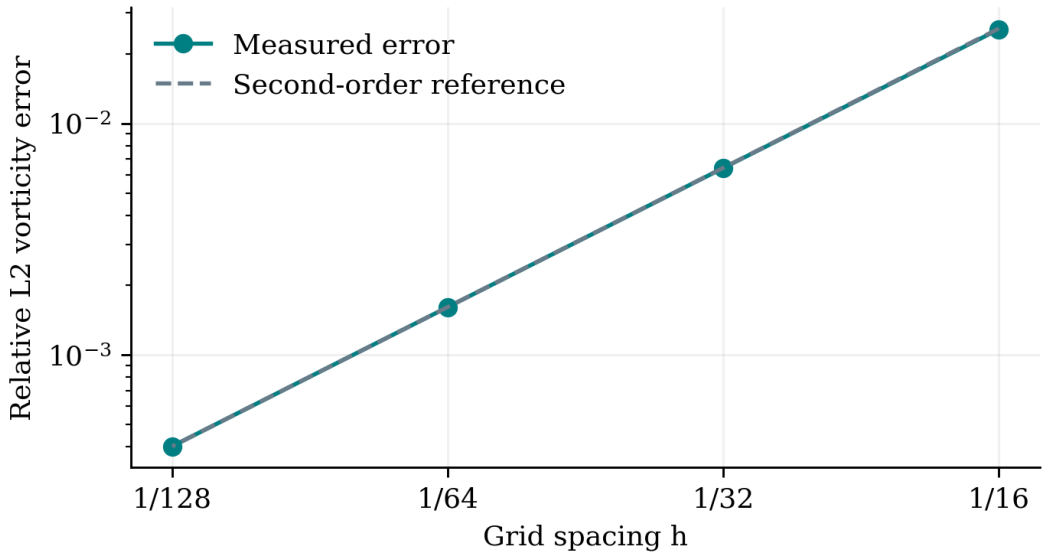


Figure 1. Error decreases at approximately second order. The reported order, 1.9965, is the slope fitted to all four log-error/log-spacing pairs.

The observed trend agrees closely with expected second-order behavior. At the finest resolution the relative vorticity error is approximately 4.015e-4, or 0.04015%. Divergence RMS stays near roundoff for this symmetric analytic construction. That tiny residual cannot replace the vorticity test: cancellation can enforce one identity while another derivative quantity is wrong.

7. Auditing the retained cavity field

We next load the complete accepted Re = 100 case from the Week-1 archive. It has a 65 by 65 grid. The upper wall moves horizontally with unit speed and the other walls are stationary. Unlike the analytic field, this flow contains a moving-lid corner incompatibility and spatially varying numerical errors. It provides a realistic input for checking the diagnostic's interpretation of stored data.

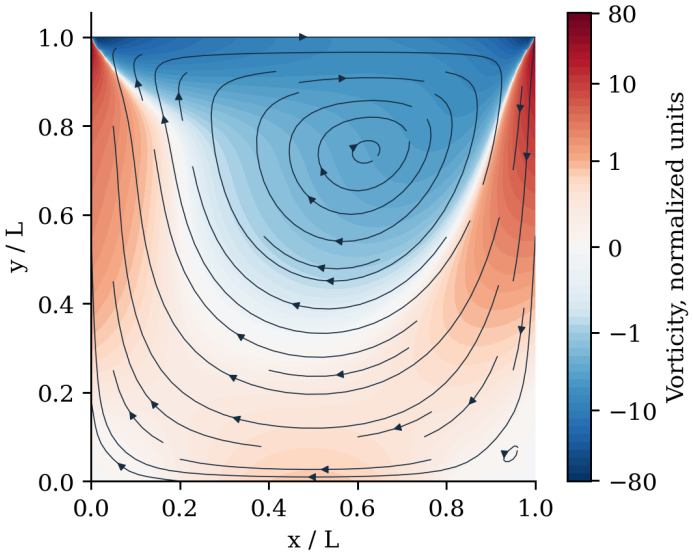


Figure 2. Vorticity computed from retained Re = 100 cavity velocities, with streamlines. Axes have equal scale. Color uses a symmetric logarithmic scale, linear within +/-1, to retain wall extremes while revealing the interior structure. Metrics use the support described in the text.

Interior divergence RMS is approximately 2.418e-16. This is a consistency result for a field generated by the streamfunction formulation, where a divergence-free velocity construction is expected. It does not imply that

momentum equations were solved to that accuracy or that the flow is experimentally validated. A post-processing residual measures the particular relation being evaluated.

The relative L2 disagreement between newly differentiated velocity and archived vorticity is approximately 2.902%. These quantities arise through different discrete operations, so exact agreement is not expected. We use the comparison to audit signs, axes and discrete support. The percentage must not be described as error against an exact cavity solution, nor as the prediction error of an AI model.

Maximum wall-velocity error is zero at stored precision for the non-corner nodes checked by the routine. Zero is appropriate because boundary values are imposed directly. This check protects the boundary convention, while the interior tests address other aspects of the field. Combining them gives more useful evidence than a convincing contour alone.

8. From measurements to an acceptance decision

Every gate in Table 2 must pass. The thresholds are fixed operational criteria in the repository, not universal physical constants or experimentally derived uncertainty bounds. Keep them fixed while assessing a candidate implementation. If the scientific question changes, explain and version the revised criteria before evaluating the new claim.

Table 2. Retained measurements and fixed criteria. All seven checks pass.

Acceptance check	Observed	Required
Dataset identity	Exact hash match	Exact match
Four-grid fitted order	1.9965	At least 1.90
Finest vorticity error	4.015e-04	At most 5e-4
Analytic divergence RMS	2.748e-14	At most 1e-12
Cavity divergence RMS	2.418e-16	At most 1e-12
Cavity vorticity disagreement	2.902e-02	At most 4e-2
Non-corner wall error	0.0e+00	At most 1e-12

The 1.90 minimum order tests whether convergence is approximately second order. The finest-grid bound separately prevents acceptance based only on a favorable rate. The divergence and wall tolerances serve the float64 construction and stored normalization. The 4% cavity-vorticity threshold is a consistency tolerance for this archive and support, not a general CFD accuracy requirement.

The dataset gate compares the archive's SHA-256 digest with its expected value. A mismatch rejects the record even if the numbers look plausible. The digest establishes file identity; it does not establish that data are physically correct or suitable for a different research question. All seven gates pass for the reference implementation. Acceptance does not extend automatically to shocks, turbulence, other boundary treatments or unstructured meshes.

9. A worked failure: swapping derivative axes

Suppose a proposed implementation calculates the following quantity and labels it vorticity:

$$\omega_{\text{wrong}} = D_y v - D_x u \neq D_x v - D_y u \tag{8}$$

On a square grid this expression returns the expected shape and finite values, so basic execution tests may pass. But it is the difference of normal derivatives, whereas z-vorticity requires cross derivatives. The program answers a different mathematical question.

On the 129-point analytic field, this faulty implementation has relative L2 error approximately 1.232, compared with the allowed 5e-4. The decision is rejection. Retaining the example demonstrates that verification detects at least one meaningful semantic fault. A reviewer should trace variable meanings through code rather than inspect only formatting and green test indicators.

Correct the derivative mapping and repeat the same tests. Increasing the threshold until the implementation passes would change the contract to accommodate a bug. A threshold change needs an independently explained change in the intended problem, numerical method or accepted uncertainty.

10. A practical workflow with an AI assistant

Start with the specification: quantity, storage convention, units, sign, support and failure behavior. A useful request is to implement divergence and z-vorticity for u[y,x] and v[y,x], use second-order finite differences on increasing coordinates, validate inputs, and preserve the acceptance criteria. This gives the assistant a concrete task whose result can be inspected.

Review returned code before relying on its output. Trace each derivative from its definition to its array axis. Check boundary handling, coordinate spacing and whether input validation silently modifies data. An assistant's explanation can help, but verify it against code and reference results. A fluent explanation is not independent evidence.

Run the analytic study before interpreting the real-case audit. Retain failed proposals when they reveal a scientific failure mode. If a gate fails, distinguish an implementation defect from an input problem or an inappropriate scientific assumption: each requires a different correction. Record the assistance used and the human corrections that affected the result. The investigator remains responsible for the claim regardless of who drafted the code.

11. Reproducing and reporting the result

The accompanying notebook is W1_1_AI_Assisted_Scientific_Software.ipynb in notebooks/week01_1. The specification is stored beside it, and acceptance_record.json is retained under results/week01_1_scientific_software. From the repository root, run the material builder to execute the notebook and reproduce its evidence and this lecture:

```
python qa/build_week01_1_materials.py --execute
```

Use a supported Python version and record the packages actually used. Preserve the repository revision and input digest with the command. Keep notebook working outputs separate from retained evidence; the dedicated builder is the deliberate regeneration path. Review the resulting differences before publishing changed evidence.

Numerical agreement and byte-identical files are different properties. A plotting-library change may alter image metadata without changing scientific values. Conversely, an unchanged image does not prove that it came from the stated input. Inspect the numerical record and execution context as well as the rendered document. Reproducibility is a chain of identifiable operations, not just a fixed random seed [1,2].

12. Required GitHub implementation assignment

The worked notebook prepares you for an actual contribution. Add net_volume_flux(x, y, u, v) in flowmllab/student_mass_balance.py using a coding agent of your choice. Compute the outward boundary integral with composite trapezoidal quadrature: integrate u on the right minus the left edge, and v on the top minus the bottom edge. For dimensional velocities this is volume flux per unit depth, in square metres per second; its normalized value is Q/(U L). It is not a variable-density mass-flux diagnostic. Zero integrated flux can hide compensating local errors, so it complements rather than replaces the divergence audit.

Follow ASSIGNMENT.md in notebooks/week01_1. Complete the supplied SPEC.md with the physical problem, inputs and outputs, units, validity limits, numerical criteria and failure cases. Commit it before requesting code, and record that commit in PROCESS_LOG.md. Preserve the actual prompt, tool and model identifier, original generated patch, edited files and human corrections. The assessed task requires an agent; any access exception must be agreed with the instructor and recorded explicitly.

Report four distinct levels of evidence: unit tests for signs and invalid inputs; numerical regression against retained Week 1.1 evidence; physical invariants on constant velocity and the closed cavity; and independent analytic boundary integrals. The instructor checker also adds 0.1 to the right-edge horizontal velocity on the unit square, requiring a flux increase of 0.1. This deliberately altered field is a fault-detection test, not a new CFD solution. A function that always returns zero must fail. Run the candidate checker from the repository root and retain its complete output:

```
python qa/check_week01_1_candidate.py flowmllab/student_mass_balance.py
```

Review every changed code and test line using LINE_REVIEW.md, including the meaning of axes, normal signs, quadrature weights, units and reference values. Submit a PR in your own fork or the instructor-designated repository with the specification-first commit, implementation, tests, process log, review and REPORT.md. Its title must be: What the agent produced, what the physicist corrected, and why. Record real corrections; if none were needed, explain the evidence supporting that conclusion. Running the reference notebook alone does not complete this assignment.

13. Exercises and assessment

Exercise 1. Derive the rigid-rotation result u = -a y, v = a x by hand. Explain which sign and axis errors it reveals. Why does a method that is exact for a linear field still need the nonlinear reference and the grid study?

Exercise 2. Use Table 1 to estimate the three adjacent two-grid orders. Compare them with the four-grid fitted order. Explain why the estimators need not be identical even when both are consistent with a second-order method.

Exercise 3. Introduce the axis swap in a copy of the diagnostic. Predict which execution checks will pass and identify the scientific gate that rejects it. Preserve the reference implementation and retained evidence.

Exercise 4. Propose a verification study for stretched Cartesian coordinates. State how you would refine the mesh, define a suitable norm and distinguish interior from boundary accuracy. Explain why API support for nonuniform coordinates is not itself evidence of convergence.

Exercises 1-4 are preparation or optional extensions. The assessed submission is the new boundary-flux function, independent tests, specification-first commit, recorded agent interaction, retained test outcomes, line review and report required by ASSIGNMENT.md. Allocate 20% to specification, 25% to verification of the flux function, 20% to physical and analytic flux checks, 15% to scientific line review, 10% to

reproducibility evidence and 10% to claim limits. Retain genuine failures; do not manufacture an error when no correction was needed.

14. Guidance for interpreting the exercises

For rigid rotation, differentiating v = a x with respect to x gives a, while differentiating u = -a y with respect to y gives -a. Subtraction therefore gives 2a. Both normal derivatives vanish, giving zero divergence. The swapped expression also gives zero in this case, which immediately exposes its failure as a vorticity diagnostic when a is nonzero. A centered difference differentiates linear functions exactly apart from roundoff, so this example tests semantics rather than the expected truncation-error rate. The nonlinear streamfunction supplies the curvature and higher derivatives needed to examine that rate.

In the grid study, each adjacent error ratio is close to four. Taking its logarithm and dividing by log(2) therefore gives an order close to two. The four-grid regression combines information from every resolution; it should be reported as such. Differences between the pairwise estimates can reflect higher-order terms and the changing physical extent of the retained interior. Do not round an observed rate to exactly two before checking the actual error values.

For a stretched mesh, first define a smooth coordinate mapping and refine its computational coordinate consistently. Evaluate the analytic field on the resulting physical coordinates, then use those coordinates in the derivative routine. A volume-weighted error norm may be preferable if the intention is to approximate a physical-domain integral: an unweighted point norm gives dense regions more influence. State whether refinement changes the physical audit region, and assess boundary accuracy separately. This proposal extends the verification method; it is not an additional retained result in the present lecture.

References and source acknowledgment

[1] Wilson, G., et al. (2017). Good Enough Practices in Scientific Computing. PLOS Computational Biology, 13(6), e1005510. https://doi.org/10.1371/journal.pcbi.1005510

[2] Sandve, G. K., et al. (2013). Ten Simple Rules for Reproducible Computational Research. PLOS Computational Biology, 9(10), e1003285. https://doi.org/10.1371/journal.pcbi.1003285

[3] Roache, P. J. (1998). Verification and Validation in Computational Science and Engineering. Hermosa Publishers.

The public Stanford CS146S overview (https://themodernsoftware.dev/) inspired the syllabus-level topic of agent-assisted development. The text, analytic example, code and figures were independently developed for FlowMLLab. The cavity comes from the retained Week-1 archive; the analytic field is explicitly constructed in this laboratory. Neither is a language-model-generated substitute for CFD data.